

EV Charging Sessions Data Engineering and Analytics Platform

Team: Durga Prasad Sunkara, Vamsi, Veera, Yashas

Date: 2026-02-13

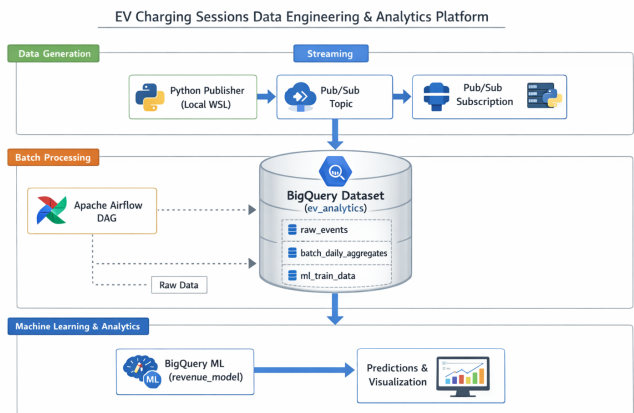
1. Project Introduction

This project implements an end-to-end data engineering and analytics pipeline for EV charging sessions. We simulate charging-session events, ingest them through a streaming layer, store and transform the data in a cloud warehouse, and train a machine learning regression model to predict total revenue. The goal is to demonstrate a complete workflow: data generation → streaming → batch aggregation → analytics → ML training → prediction and visualization.

2. Architecture Overview

The pipeline has four logical layers:

- Data Generation: Python scripts generate realistic EV session events.
- Streaming: Events flow through Kafka (local) and Google Pub/Sub (cloud).
- Warehouse: Google BigQuery stores curated tables and aggregates.
- ML & Analytics: BigQuery ML trains a regression model and SQL provides insights and visualizations.



3. Technology Stack

- Languages: Python, SQL
- Streaming: Apache Kafka (local), Google Pub/Sub (cloud)
- Orchestration / Batch: Apache Airflow (DAG-based batch aggregations)
- Warehouse & Analytics: Google BigQuery
- ML: BigQuery ML (Linear Regression)
- Infrastructure: Docker Compose, WSL (Ubuntu on Windows)
- Visualization: BigQuery Studio charts (scatter/line)

4. Dataset (EV Charging Sessions)

Each generated event represents a charging activity with the fields below. This schema is simple but realistic enough to support aggregations and ML features.

- event_time (timestamp)
- session_id (string)
- station_id (string)
- city (string)
- charger_type (string: fast/normal)
- energy_kwh (float)
- duration_min (float)
- price_eur (float)
- event_type (string: start/stop)

The dataset was produced continuously so that streaming consumers and batch jobs could be validated in real time.

5. Streaming Layer (Kafka + Pub/Sub)

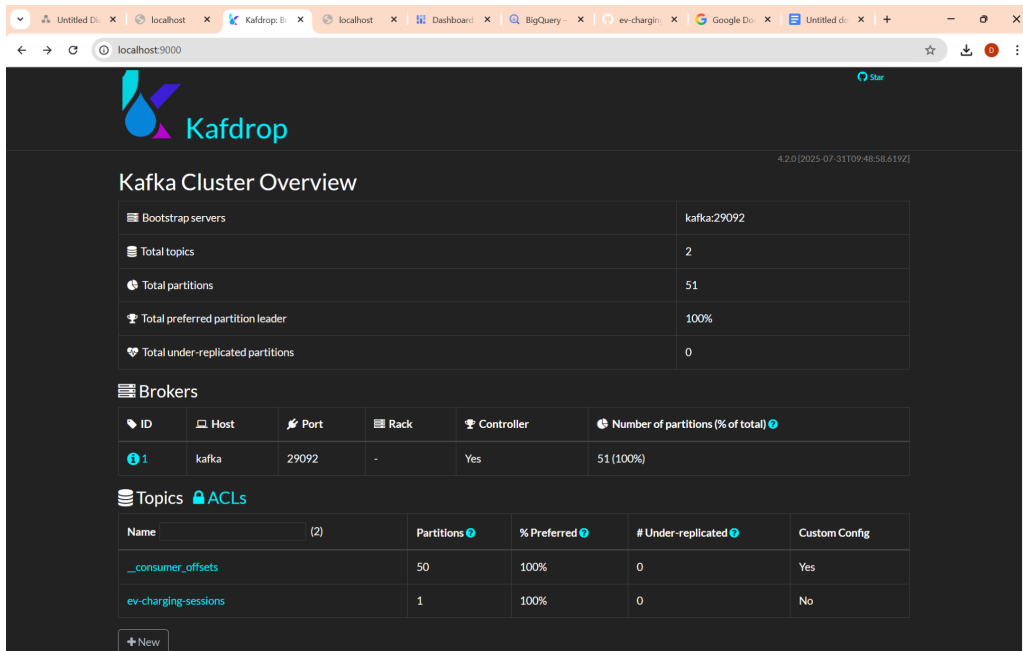
We used Kafka locally to validate a classic streaming setup and used Pub/Sub in Google Cloud to connect the data generator to cloud ingestion. Kafka was started using Docker Compose (Zookeeper + Kafka broker + Kafdrop UI). Pub/Sub publisher/consumer scripts were executed from the Python virtual environment.

```

sunkara@sunkara:~$ pip install -U pip
Requirement already satisfied: pip in ./env/lib/python3.12/site-packages (26.0.1)
(.venv) sunkara@sunkara:~/DurgaprasadSunkara: /mnt/c/ev-charging-sessions$ docker compose down
[+] Running 4/4
  Container ev-charging-sessions-kafdrop-1      Removed      3.0s
  Container ev-charging-sessions-kafka-1        Removed      2.6s
  Container ev-charging-sessions-zookeeper-1    Removed      1.1s
  Network ev-charging-sessions-default          Removed      1.0s
(.venv) sunkara@sunkara:~/DurgaprasadSunkara: /mnt/c/ev-charging-sessions$ docker-compose up -d
docker ps
[+] Running 4/4
  Network ev-charging-sessions-default          Created      0.8s
  Container ev-charging-sessions-zookeeper-1    Started      1.7s
  Container ev-charging-sessions-kafka-1        Started      1.6s
  Container ev-charging-sessions-kafdrop-1      Started      1.7s
CONTAINER ID   IMAGE                                NAMES                                CREATED      STATUS      PORTS
82e5b5bf49154 imageindynamics/kafdrop             "/kafdrop.sh"                      2 seconds ago Up Less than a second 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp
800923bd3b52a confluentinc/cp-kafka-1.5.0         "/etc/confluent/docker."          2 seconds ago Up Less than a second 0.0.0.0:9092->9092/tcp, [::]:9092->9092/tcp
67b9b6b6aed1 confluentinc/cp-zookeeper-7.5.0      "/etc/confluent/docker."          2 seconds ago Up 1 second      2888/tcp, 0.0.0.0:2181->2181/tcp, [::]:2181->2181/tcp
(.venv) sunkara@sunkara:~/DurgaprasadSunkara: /mnt/c/ev-charging-sessions$ ls -l streaming
total 12
-rwxrwxrwx 1 sunkara sunkara 2516 Feb 3 12:23 consumer.py
-rwxrwxrwx 1 sunkara sunkara 610 Feb 11 13:37 consumer_pubsub.py
-rwxrwxrwx 1 sunkara sunkara 1260 Feb 13 01:42 publisher_pubsub.py
(.venv) sunkara@sunkara:~/DurgaprasadSunkara: /mnt/c/ev-charging-sessions$ python streaming/consumer.py
consumer started
C:\Traceback (most recent call last):
  File "D:\mnt/c/ev-charging-sessions/streaming/consumer.py", line 82, in <module>
    main()
  File "D:\mnt/c/ev-charging-sessions/streaming/consumer.py", line 36, in main
    for msg in consumer:
  File "D:\mnt/c/ev-charging-sessions/.venv/lib/python3.12/site-packages/kafka/consumer/group.py", line 1213, in __next__
    return next(self._iterator)
  File "D:\mnt/c/ev-charging-sessions/.venv/lib/python3.12/site-packages/kafka/consumer/group.py", line 1185, in _message_generator_v2
    record_map = self.poll(timeout_ms=timeout_ms, update_offsets=False)
  File "D:\mnt/c/ev-charging-sessions/.venv/lib/python3.12/site-packages/kafka/consumer/group.py", line 708, in poll
    records = self._poll_once(timer, max_records, update_offsets=update_offsets)
  File "D:\mnt/c/ev-charging-sessions/.venv/lib/python3.12/site-packages/kafka/consumer/group.py", line 757, in _poll_once
    self._client.poll(timeout_ms=poll_timeout_ms)

```

Docker containers running (Kafka/Zookeeper/Kafdrop)



Kafdrop UI showing broker and topic(s) on localhost:9000

```

(.venv) sunkara@DurgaprasadSunkara:/mnt/c/ev-charging-sessions$ cd /mnt/c/ev-charging-sessions
source .venv/bin/activate
export GOOGLE_APPLICATION_CREDENTIALS="$(pwd)/gcp-key.json"
python streaming/consumer_pubsub.py
Listening on: projects/ev-charging-platform-487012/subscriptions/ev-charging-sub
Received: {'event_time': '2026-02-13T01:42:48.663024+00:00', 'session_id': 'sess_test_001', 'station_id': 'station_1', 'city': 'Hamburg', 'charger_type': 'fast', 'energy_kwh': 22.5, 'duration_min': 35, 'price_eur': 9.99, 'event_type': 'start'}
(.venv) sunkara@DurgaprasadSunkara:/mnt/c/ev-charging-sessions$ python streaming/consumer_pubsub.py
Listening on: projects/ev-charging-platform-487012/subscriptions/ev-charging-sub
Received: {'event_time': '2026-02-13T03:37:34.447053+00:00', 'session_id': 'sess_test_001', 'station_id': 'station_1', 'city': 'Hamburg', 'charger_type': 'fast', 'energy_kwh': 22.5, 'duration_min': 35, 'price_eur': 9.99, 'event_type': 'start'}

```

Pub/Sub consumer receiving events

6. Batch Processing (Airflow) and Aggregations

Batch processing was implemented using an Airflow DAG in the batch_airflow/ folder. The DAG is responsible for:

- Reading curated/ingested events in BigQuery (or loading raw files depending on the run).
- Producing daily aggregates used for analytics and model training.

- Creating or replacing the BigQuery table `ev_analytics.batch_daily_aggregates`.

The screenshot shows the Google Cloud BigQuery console interface. A query has been executed, and the results are displayed in a table. The query is: `SELECT * FROM 'ev-charging-platform-487012.ev_analytics.revenue_dashboard'`. The results table has columns: `unique_sessions`, `avg_energy_kwh`, `avg_duration_min`, `actual_revenue`, and `predicted_revenue`. The table contains 14 rows of data.

Row	unique_sessions	avg_energy_kwh	avg_duration_min	actual_revenue	predicted_revenue
1	1	53.6	80.0	25.33	19.5873965769...
2	1	53.25	14.0	25.85	18.42528481895...
3	1	29.14	35.0	10.87	12.50721812239...
4	1	47.41	35.0	7.75	17.24608193832...
5	1	16.6	82.0	17.95	9.98988893380...
6	1	42.17	64.0	5.78	16.35285313808...
7	1	23.82	20.0	8.41	10.86296712304...
8	1	5.52	86.0	3.03	7.171029743631...
9	1	41.89	38.0	29.94	15.85809440113...
10	1	27.78	29.0	28.89	12.03966728996...
11	1	48.27	30.0	18.43	17.38877872195...
12	1	47.54	77.0	21.25	17.96143746907...
13	1	50.09	62.0	19.25	18.38171978959...
14	1	46.76	34.0	28.8	17.06068126269...

Airflow UI: DAG graph

7. Cloud Warehouse (BigQuery) Tables

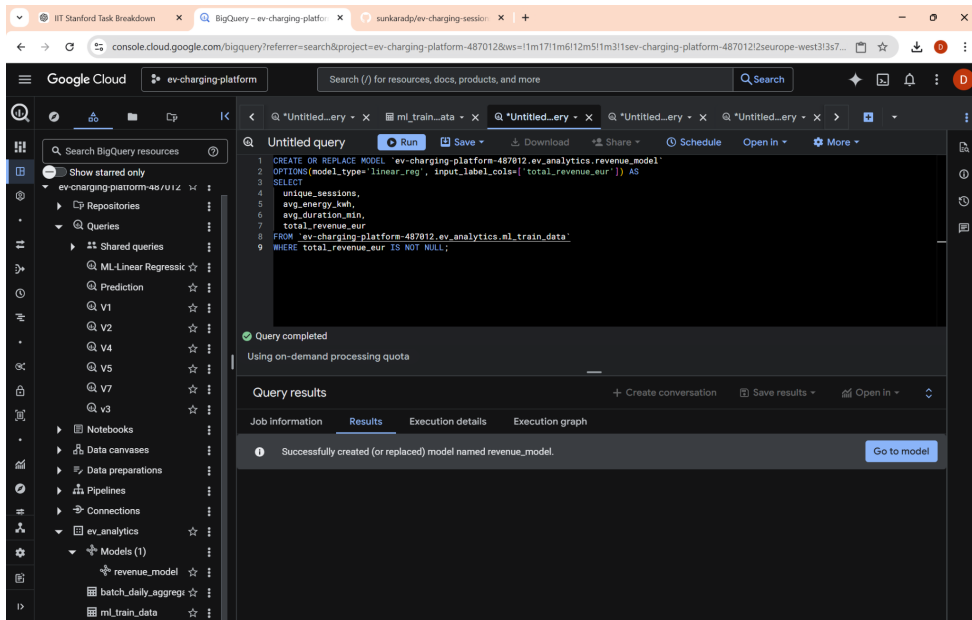
We used BigQuery as the central warehouse. Key objects created in dataset `ev_analytics`:

- `batch_daily_aggregates`: batch output table used for reporting and ML features.
- `ml_train_data`: final training dataset (features + label).
- `revenue_model`: BigQuery ML model.

The screenshot shows the Google Cloud BigQuery console interface. The 'Recent' section displays a list of objects created in the `ev_analytics` dataset. The objects are:

Display name	Type	Last modified time	Project
<code>batch_daily_aggregates</code>	Table	Feb 13, 2026, 9:57:18 AM	ev-charging-platform-487012
<code>ev_analytics</code>	Dataset	Feb 13, 2026, 9:57:15 AM	ev-charging-platform-487012
<code>Prediction</code>	Query	Feb 13, 2026, 2:57:54 AM	ev-charging-platform-487012
<code>revenue_model</code>	Model	Feb 13, 2026, 2:55:50 AM	ev-charging-platform-487012
<code>ML_Linear Regression</code>	Query	Feb 13, 2026, 2:55:47 AM	ev-charging-platform-487012
<code>V7</code>	Query	Feb 13, 2026, 2:55:12 AM	ev-charging-platform-487012
<code>V5</code>	Query	Feb 13, 2026, 2:54:58 AM	ev-charging-platform-487012
<code>V4</code>	Query	Feb 13, 2026, 2:54:38 AM	ev-charging-platform-487012
<code>V3</code>	Query	Feb 13, 2026, 2:54:23 AM	ev-charging-platform-487012
<code>V2</code>	Query	Feb 13, 2026, 2:54:10 AM	ev-charging-platform-487012

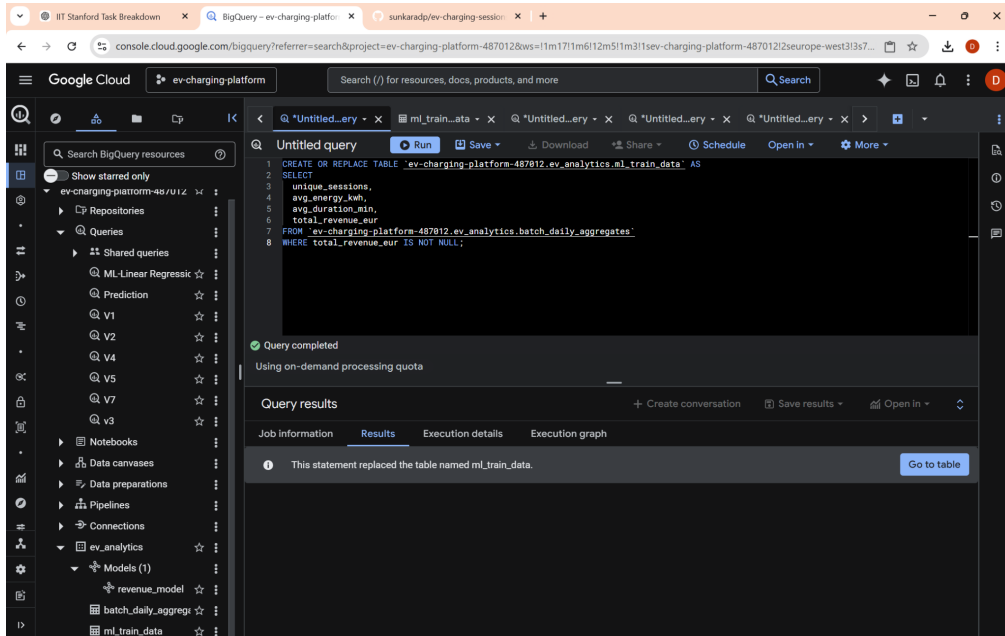
BigQuery dataset (`ev_analytics`) and objects



ml_train_data schema/preview (table view)

8. Feature Engineering (SQL) → ml_train_data

We created the ML-ready dataset by selecting the required columns from the batch aggregate table. This keeps the ML input stable and makes model retraining repeatable.



ml_train_data (SQL)

9. Machine Learning (BigQuery ML)

We trained a linear regression model in BigQuery ML to predict total_revenue_eur using:

- unique_sessions
- avg_energy_kwh
- avg_duration_min

The model is trained and stored as ev_analytics.revenue_model. Training and evaluation are executed directly via SQL, and the BigQuery UI provide training provide graph

The screenshot shows the Google Cloud BigQuery ML interface. The left sidebar displays the project structure, including the 'ev_analytics' dataset and the 'revenue_model' model. The main panel shows the 'ml_train_data' table schema with the following columns:

Field name	Type	Mode	Description	Key	Collation	Default Value	Policy Tags	Data Policy
unique_sessions	INTEGER	NULLABLE		-	-	-	-	-
avg_energy_kwh	FLOAT	NULLABLE		-	-	-	-	-
avg_duration_min	FLOAT	NULLABLE		-	-	-	-	-
total_revenue_eur	FLOAT	NULLABLE		-	-	-	-	-

revenue_model (BigQuery ML)

The screenshot shows the Google Cloud BigQuery ML interface with the 'revenue_model' selected. The main panel displays the SQL query used for training and evaluation:

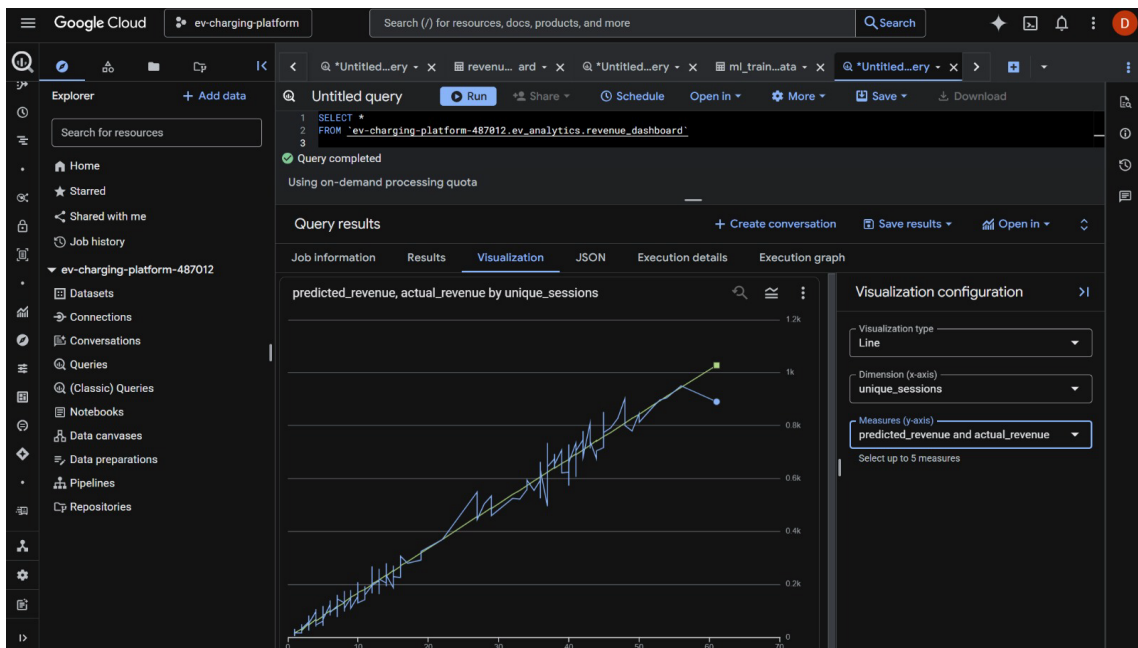
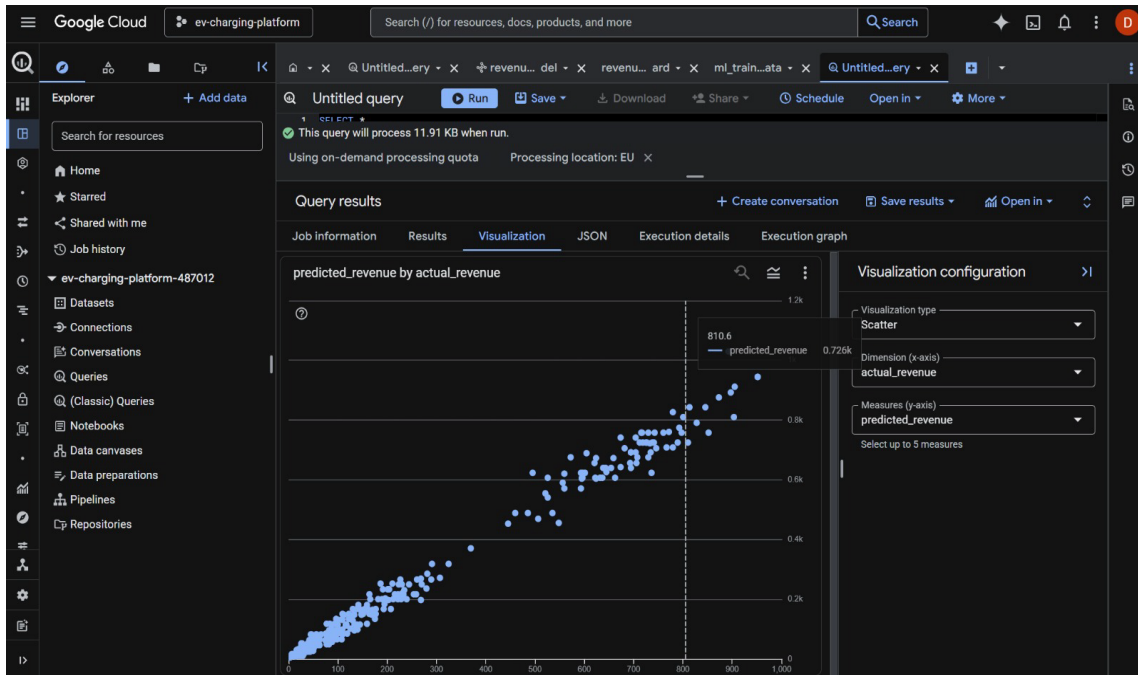
```
1 SELECT
2   unique_sessions,
3   avg_energy_kwh,
4   avg_duration_min,
5   total_revenue_eur AS actual_total_revenue_eur,
6   predicted_total_revenue_eur
7 FROM ML.PREDICT(
8   MODEL `ev-charging-platform-487812.ev_analytics.revenue_model`,
9   (
10    SELECT
11      unique_sessions,
12      avg_energy_kwh,
13      avg_dur
14    FROM ev-charging-platform-487812.ev_analytics.ml_train_data
15   )
16  LIMIT 10
```

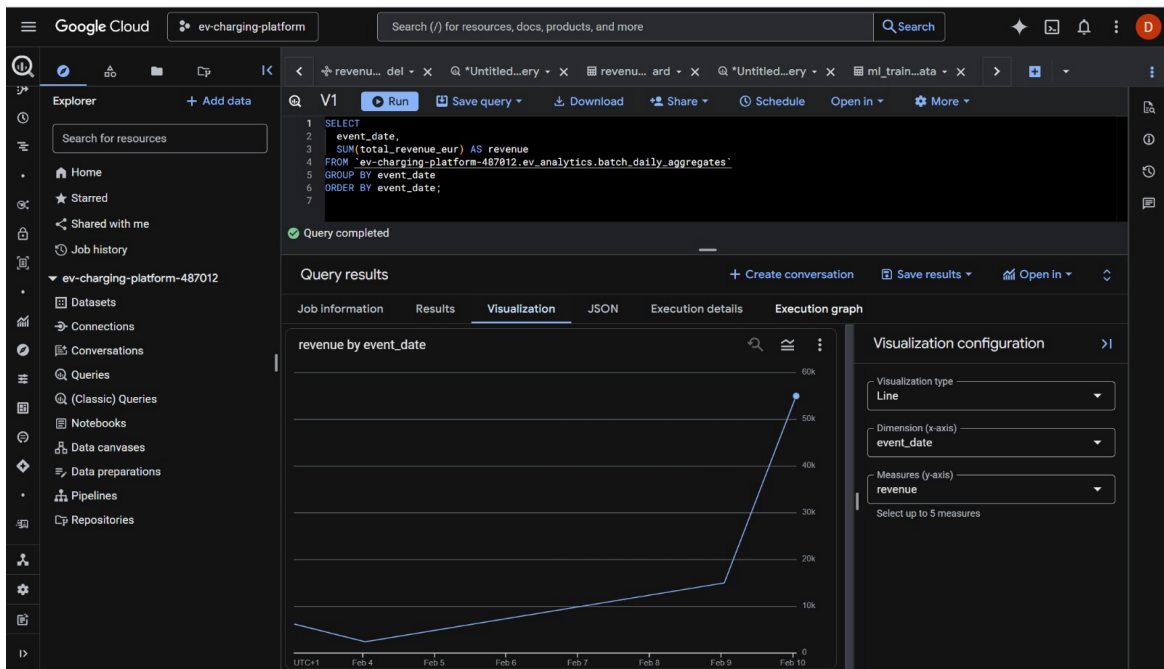
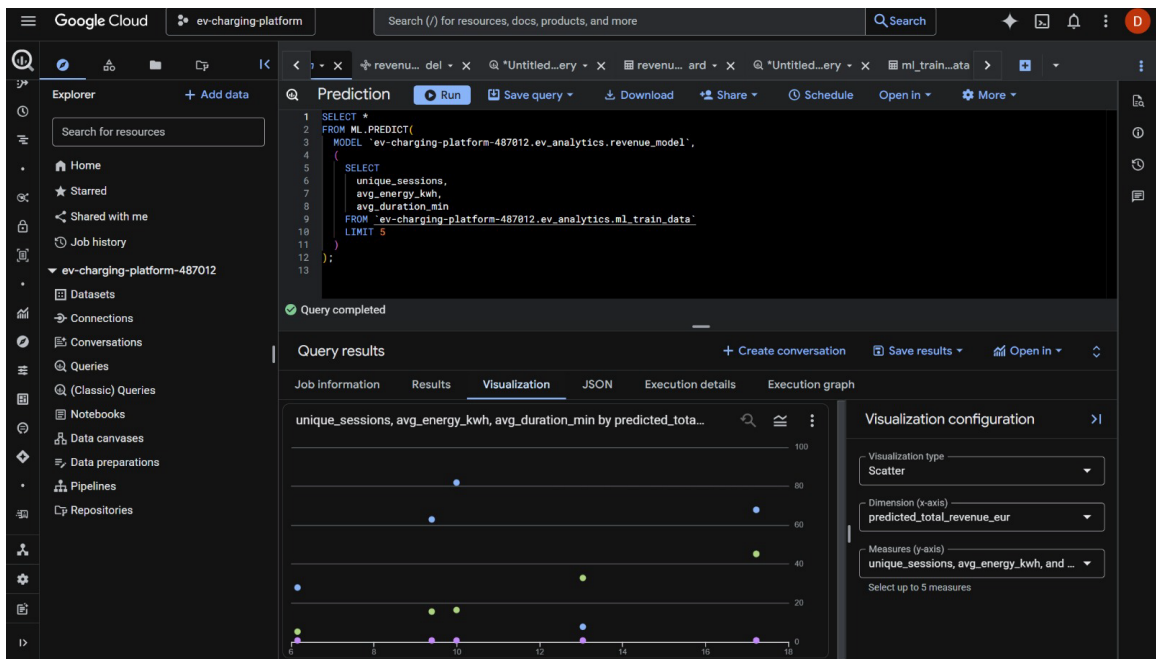
The 'Query results' section shows a visualization of the training metrics. The x-axis is labeled 'unique_sessions' and the y-axis is labeled 'avg_energy_kwh, avg_duration_min, act...'. The visualization type is set to 'Line'.

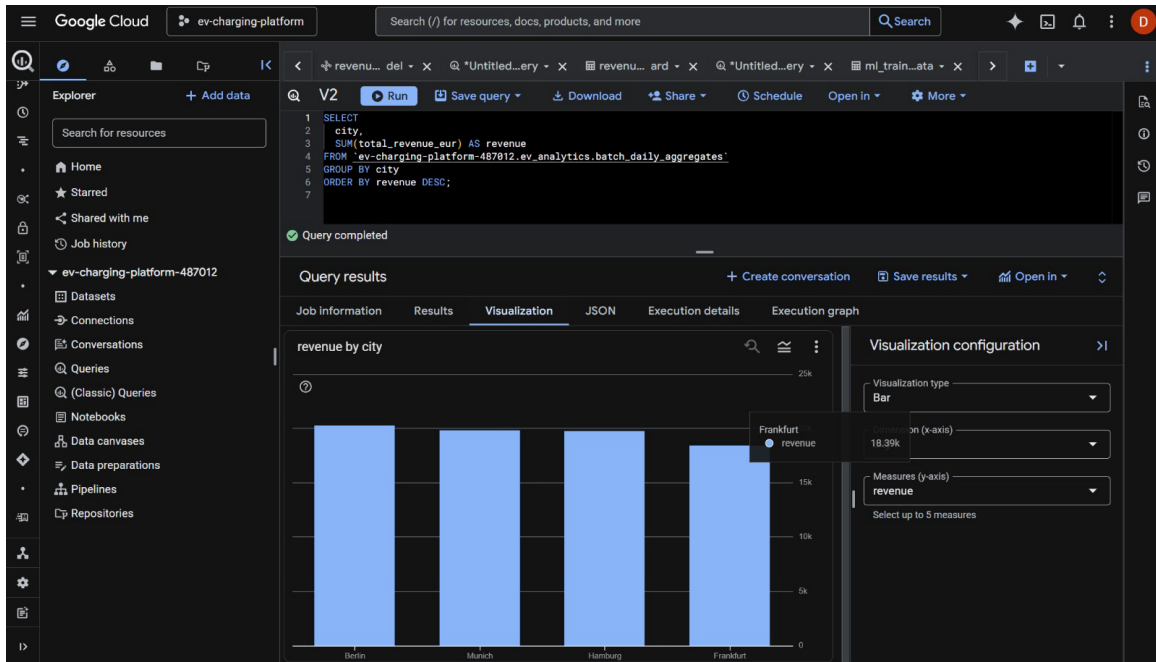
Training metrics (Loss graph)

10. Predictions and Visualization (SQL)

Predictions were generated using ML.PREDICT and compared with actual total revenue from the table. BigQuery Studio charts were used for quick visualization (scatter/line).

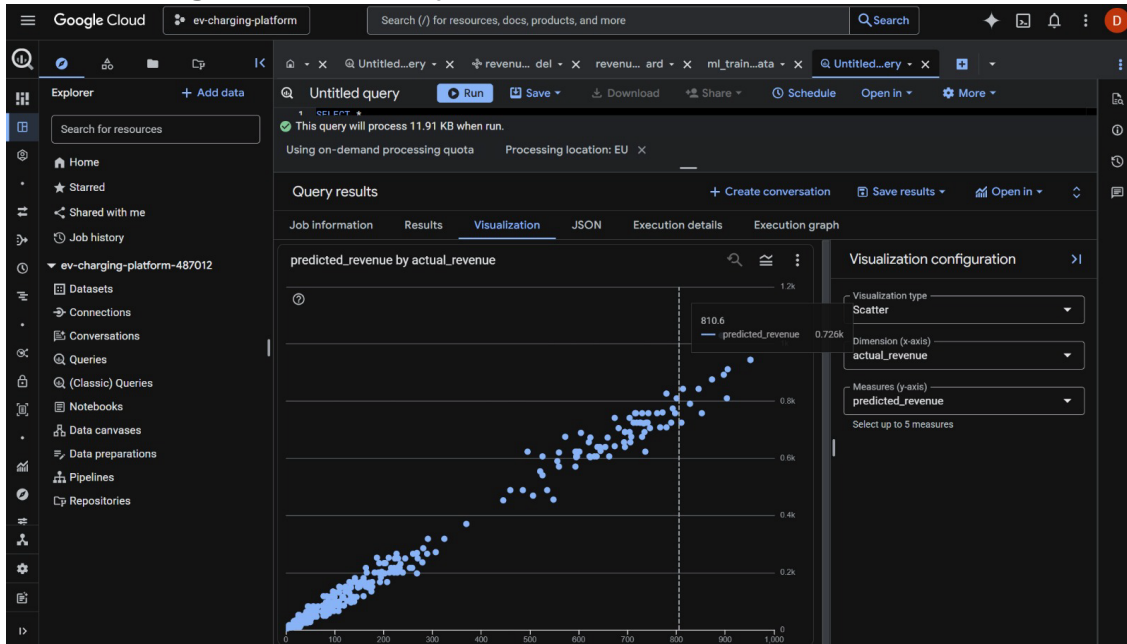






This screenshots are Prediction results table (rows)

11. Monitoring and Observability



Operational monitoring in our demo was primarily verified via:

- Docker container health (docker ps) for Kafka stack.
- Kafdrop UI to confirm topics/partitions.
- Pub/Sub consumer logs to confirm message flow.

- BigQuery job history and query status for batch + ML.

12. Issues Faced and Fixes

1. Wrong working directory / missing file path

Example: running `python streaming/producer.py` when the file did not exist. Fixed by listing files and running the correct script name and path (e.g., `python streaming/publisher_pubsub.py`).

2. Pub/Sub authentication (ADC) errors

Publisher/consumer failed when `GOOGLE_APPLICATION_CREDENTIALS` was not set or key file path was wrong. Fixed by exporting `GOOGLE_APPLICATION_CREDENTIALS=$(pwd)/gcp-key.json` before running Pub/Sub scripts.

3. Python env mismatch

Pub/Sub import errors occurred when scripts were run inside a different virtual environment that didn't have `google-cloud-pubsub` installed. Fixed by installing dependencies in the active venv (`pip install google-cloud-pubsub`).

4. Schema mismatch in SQL

A query referenced `price_eur` in `batch_daily_aggregates`, but the table contained `total_revenue_eur`. Fixed by using the correct column name.

5. Kafka consumer waiting / timeout

If the consumer starts before producer publishes, it may appear to 'hang'. Fixed by starting producer first or increasing logs and timeouts.

13. Team Contributions (4 Members)

We divided responsibilities to keep development parallel and to match the required components.

- Durga: Project integration, cloud setup (BigQuery dataset/tables), BigQuery ML model training + prediction SQL, final README/report integration.
- Vamsi: Kafka + Docker Compose setup, topic verification in Kafdrop, local streaming validation and troubleshooting.
- Veera: Batch pipeline orchestration with Airflow DAG, daily aggregation logic, and scheduling/verification steps.
- Yashas: SQL analytics and visualizations in BigQuery Studio, validation queries, and final charts/screenshots preparation.

14. Conclusion

The pipeline demonstrates an end-to-end cloud data engineering workflow: real-time event generation, streaming transport, warehouse storage, batch aggregation, analytics, and ML-based revenue prediction. The resulting artifacts (tables, model, and charts) can be recreated using the commands and SQL included in the README. The design is modular and can be extended to support additional cities, more granular station analytics, and production-grade monitoring.

15.Reference

<https://github.com/sunkaradp/ev-charging-sessions.git>